

```

import pandas as pd
import numpy as np

np.random.seed(42)

n = 500

customer_id = np.arange(1, n+1)

gender = np.random.choice(
    ['Male', 'Female'],
    n
)

age = np.random.randint(
    18,
    70,
    n
)

annual_income = np.random.randint(
    15,
    140,
    n
)

spending_score = np.random.randint(
    1,
    100,
    n
)

df = pd.DataFrame({
    "CustomerID": customer_id,
    "Gender": gender,
    "Age": age,
    "Annual Income (k$)": annual_income,
    "Spending Score (1-100)": spending_score
})

df.to_csv(
    "Mall_Customers.csv",
    index=False
)

print(df.head())
print("\nDataset Shape:", df.shape)

```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	38	77	41
1	2	Female	49	112	35
2	3	Male	40	66	63
3	4	Male	50	135	25
4	5	Male	20	52	90

Dataset Shape: (500, 5)

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

```

```

df = pd.read_csv("Mall_Customers.csv")

df.head()

```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	38	77	41
1	2	Female	49	112	35
2	3	Male	40	66	63
3	4	Male	50	135	25
4	5	Male	20	52	90

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print(df.shape)
```

```
df.info()
```

```
df.describe()
```

```
(500, 5)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CustomerID                           500 non-null    int64
1   Gender                               500 non-null    object
2   Age                                   500 non-null    int64
3   Annual Income (k$)                   500 non-null    int64
4   Spending Score (1-100)               500 non-null    int64
dtypes: int64(4), object(1)
memory usage: 19.7+ KB
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
count	500.000000	500.000000	500.000000	500.000000
mean	250.500000	43.800000	74.862000	50.732000
std	144.481833	14.835504	36.760872	29.050577
min	1.000000	18.000000	15.000000	1.000000
25%	125.750000	31.000000	42.000000	25.750000
50%	250.500000	44.000000	74.000000	50.500000
75%	375.250000	56.000000	108.000000	75.250000
max	500.000000	69.000000	139.000000	99.000000

```
print(df.isnull().sum())
```

```
CustomerID      0
Gender          0
Age             0
Annual Income (k$)  0
Spending Score (1-100)  0
dtype: int64
```

```
df = df.drop_duplicates()
```

```
print("New Shape:", df.shape)
```

```
New Shape: (500, 5)
```

```
X = df[
    [
        "Age",
        "Annual Income (k$)",
        "Spending Score (1-100)"
    ]
]
```

```
]
X.head()
```

	Age	Annual Income (k\$)	Spending Score (1-100)	
0	38	77	41	
1	49	112	35	
2	40	66	63	
3	50	135	25	
4	20	52	90	

Next steps: [Generate code with X](#) [New interactive sheet](#)

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print(X_scaled[:5])

[[-0.39134556  0.05821791 -0.33533745]
 [ 0.35086154  1.01127074 -0.54208064]
 [-0.25639882 -0.24131298  0.42272091]
 [ 0.41833491  1.6375626  -0.88665262]
 [-1.60586627 -0.62253411  1.35306526]]
```

```
wcss = []

for i in range(1,11):

    kmeans = KMeans(
        n_clusters=i,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)

    wcss.append(
        kmeans.inertia_
    )

plt.figure(figsize=(8,5))

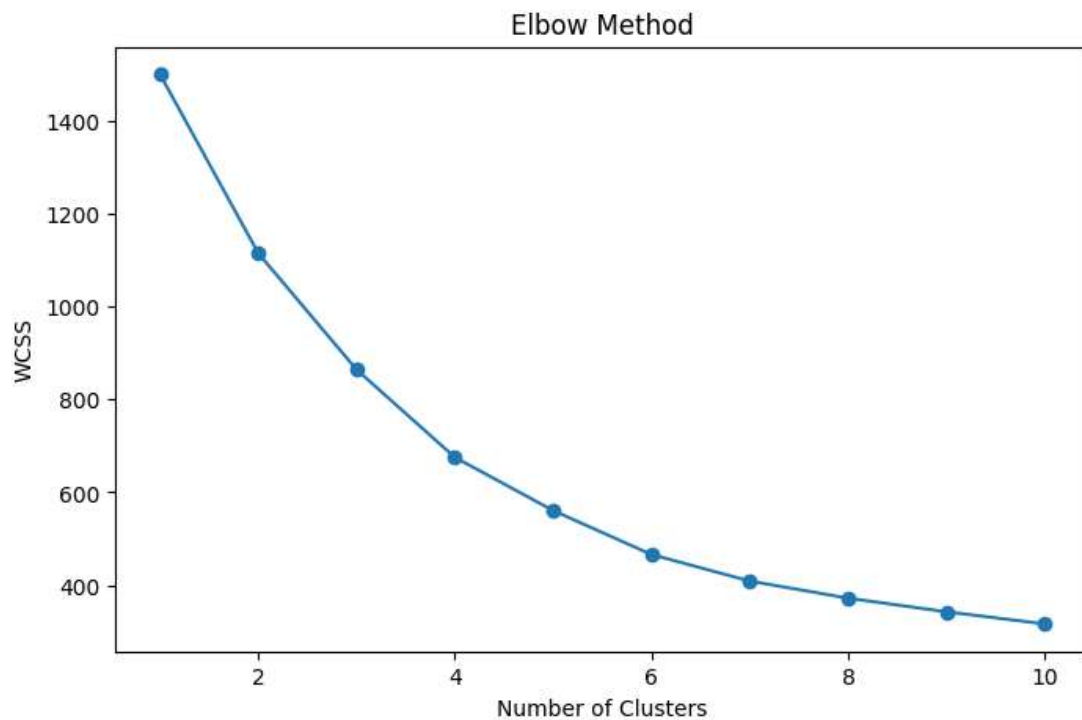
plt.plot(
    range(1,11),
    wcss,
    marker='o'
)

plt.title("Elbow Method")

plt.xlabel("Number of Clusters")

plt.ylabel("WCSS")

plt.show()
```



```
for k in range(2,11):

    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(
        X_scaled
    )

    score = silhouette_score(
        X_scaled,
        labels
    )

    print(
        f"K={k}  Silhouette Score={score:.4f}"
    )
```

```
K=2  Silhouette Score=0.2468
K=3  Silhouette Score=0.2540
K=4  Silhouette Score=0.2840
K=5  Silhouette Score=0.2913
K=6  Silhouette Score=0.3074
K=7  Silhouette Score=0.3009
K=8  Silhouette Score=0.2923
K=9  Silhouette Score=0.2939
K=10 Silhouette Score=0.2812
```

```
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(
    X_scaled
)

df["Cluster"] = clusters

df.head()
```

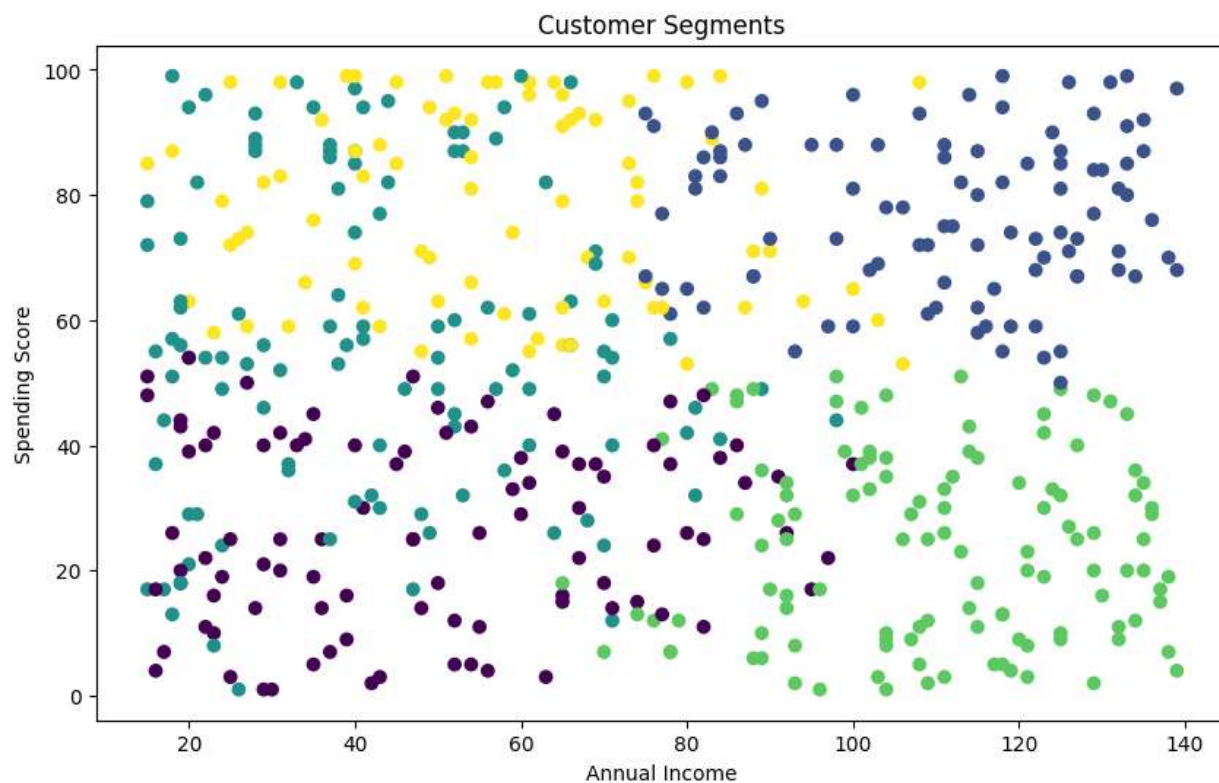
	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster	
	0	1	Male	38	77	41	3
	1	2	Female	49	112	35	3
	2	3	Male	40	66	63	2
	3	4	Male	50	135	25	3
	4	5	Male	20	52	90	2

Next steps: [Generate code with df](#)[New interactive sheet](#)

```
print(  
    df["Cluster"]  
    .value_counts()  
)
```

```
Cluster  
3    115  
2    112  
1     96  
0     95  
4     82  
Name: count, dtype: int64
```

```
plt.figure(figsize=(10,6))  
  
plt.scatter(  
    df["Annual Income (k$)"],  
    df["Spending Score (1-100)"],  
    c=df["Cluster"]  
)  
  
plt.xlabel(  
    "Annual Income"  
)  
  
plt.ylabel(  
    "Spending Score"  
)  
  
plt.title(  
    "Customer Segments"  
)  
  
plt.show()
```



```
centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)

print("Cluster Centroids:\n")

print(
    pd.DataFrame(
        centroids,
        columns=X.columns
    )
)
```

Cluster Centroids:

	Age	Annual Income (k\$)	Spending Score (1-100)
0	56.073684	48.810526	26.305263
1	39.010417	110.864583	76.968750
2	28.125000	43.160714	57.008929
3	43.486957	109.921739	23.417391
4	57.036585	57.024390	78.048780

```
plt.figure(figsize=(10,6))

plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=df["Cluster"]
)

centroids = scaler.inverse_transform(
    kmeans.cluster_centers_
)

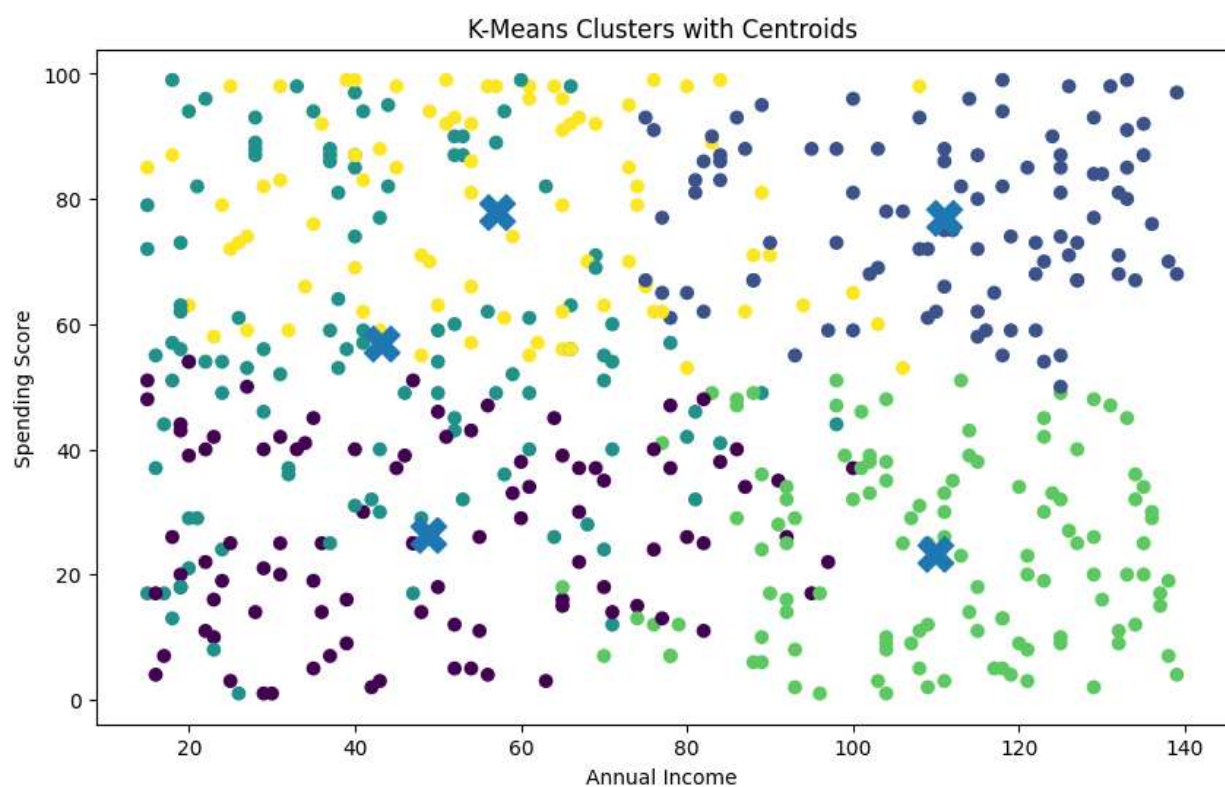
plt.scatter(
    centroids[:,1],
    centroids[:,2],
    marker='X',
    s=300
)
```

```
plt.xlabel(
    "Annual Income"
)

plt.ylabel(
    "Spending Score"
)

plt.title(
    "K-Means Clusters with Centroids"
)

plt.show()
```



```
segment_summary = df.groupby(
    "Cluster"
).mean(
    numeric_only=True
)

segment_summary
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)	
Cluster					
0	255.421053	56.073684	48.810526	26.305263	
1	240.572917	39.010417	110.864583	76.968750	
2	246.741071	28.125000	43.160714	57.008929	
3	243.808696	43.486957	109.921739	23.417391	
4	270.939024	57.036585	57.024390	78.048780	

Next steps: [Generate code with segment_summary](#)

[New interactive sheet](#)

```
for cluster in sorted(
    df["Cluster"].unique()
):

    print(
        f"\nCluster {cluster}"
    )

    print(
        "Targeted marketing campaigns can be designed for this customer segment."
    )
```

Cluster 0
Targeted marketing campaigns can be designed for this customer segment.

Cluster 1
Targeted marketing campaigns can be designed for this customer segment.

Cluster 2
Targeted marketing campaigns can be designed for this customer segment.

Cluster 3
Targeted marketing campaigns can be designed for this customer segment.

Cluster 4
Targeted marketing campaigns can be designed for this customer segment.

```
print("OBSERVATIONS")

print("-"*50)

print("1. Customers were grouped into distinct clusters.")

print("2. Income and spending behavior influenced clustering.")

print("3. High-income customers formed separate segments.")

print("4. Low-spending customers formed another segment.")
```